

# **RESOLUCIÓN DE PROBLEMAS Y ALGORITMOS**

**Dra. Jessica Andrea Carballido**

**Dpto. de Ciencias e Ingeniería de la Computación  
UNIVERSIDAD NACIONAL DEL SUR**

3

## Aclaración sobre el “write”/”writeln”

```
program abc4;  
var a: integer;  
    z: real;  
  
begin  
    writeln('Ingrese un valor para a: ');  
    readln(a);  
    z := sqrt(a);  
    writeln(z:0:2);  
    writeln(z);  
    readln;  
end.
```

```
Ingrese un valor para a:  
11  
3.32  
3.3166247903553998E+000  
|
```

El **write** **redondea** el número real a la cantidad de decimales mostrados.

# SINTAXIS



- ❖ La **sintaxis** es la rama de la gramática que estudia las **relaciones de las palabras al combinarse** para formar unidades superiores en significado.
- ❖ En **informática**, la **sintaxis** es el **grupo de normas** que marcan las secuencias correctas de los elementos propios de un lenguaje de programación.
- ❖ La sintaxis de un lenguaje de programación es entonces el conjunto de **reglas que deben seguirse al escribir el código** fuente de los programas para considerarse como correctos para ese lenguaje de programación.

CODIFICACIÓN

# Elementos de Pascal

CODIFICACIÓN

Todo programa comienza por la palabra *program* seguido de un **nombre**.

El **';**' es un separador de instrucciones.

```
program areaCuadrado;
```

```
var lado, area: integer;
```

```
begin
```

```
  write('Ingrese la longitud del lado: ');
```

```
  readln(lado);
```

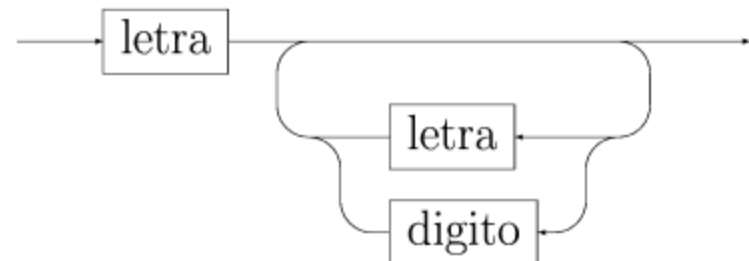
```
  area := lado * lado;
```

```
  writeln('El área del cuadrado es ', área);
```

```
end.
```

**SINTAXIS**

*identificador*



# Elementos de Pascal

CODIFICACIÓN

Los programas pueden incluir una o **más variables**.

Cada variable se DECLARA: un **nombre** y un **tipo**.

El bloque de declaraciones comienza con la palabra reservada **var**.

```
program areaCuadrado;
```

```
var lado, area: integer;
```

```
begin
```

```
  write('Ingrese la longitud de lado: ');
```

```
  readln(lado);
```

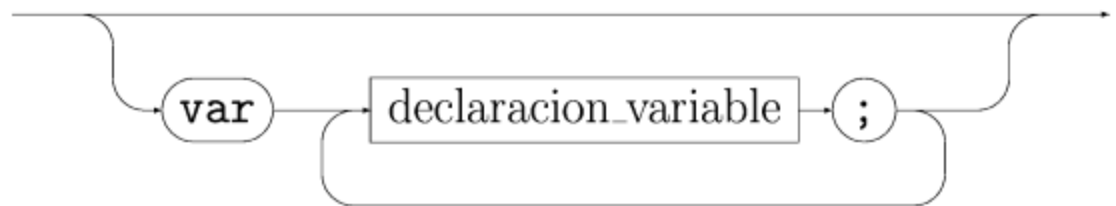
```
  area := lado * lado;
```

```
  writeln('El área del cuadrado es: ', area);
```

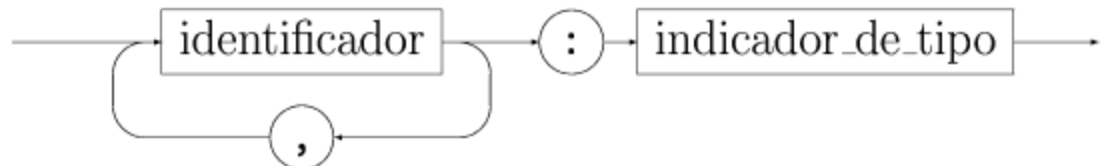
```
end.
```

**SINTAXIS**

*declaracion\_de\_variables*



*declaracion\_variable*



# Elementos de Pascal

La sección ejecutable (**acciones**) del programa se encierra siempre entre las palabras **begin – end** y termina con **'**. Se llama **BLOQUE EJECUTABLE**.

```
program areaCuadrado;
```

```
var lado, area: integer;
```

```
begin
```

```
  write('Ingrese la longitud del lado: ');
```

```
  readln(lado);
```

```
  area := lado * lado;
```

```
  writeln('El área del cuadrado es ', area);
```

```
end.
```

# Elementos de Pascal

El programa se comunica con el usuario a través de la consola, con instrucciones de entrada (read) y salida (write).

```
program areaCuadrado;
```

```
var lado, area: integer;
```

```
begin
```

```
  write('Ingrese la longitud del lado: ');
```

```
  readln(lado);
```

```
  area := lado * lado;
```

```
  writeln('El área del cuadrado es ', area);
```

```
end.
```

# Elementos de Pascal

La asignación es la instrucción **fundamental** en un lenguaje de programación *imperativo* como Pascal.

```
program areaCuadrado;  
var lado, area: integer;  
begin  
    write('Ingrese la longitud del lado: ');  
    readln(lado);  
    area := lado * lado;  
    writeln('El área del cuadrado es ', area);  
end.
```



# Elementos de Pascal

Escribir un programa en Pascal que calcule y muestre el perímetro y la superficie de un círculo a partir del valor de su radio.

¿Cuáles son los datos de entrada?

¿Alguno de los datos es *constante*?

¿Cuáles son los datos de salida?

Algoritmo variasOp

DE: radio (un número positivo)

DS: perim, área (dos números positivos)

$\text{perim} \leftarrow 3.1416 * 2 * \text{radio}$

$\text{area} \leftarrow 3.1416 * \text{radio} * \text{radio}$

# Elementos de Pascal

Escribir un programa en Pascal que calcule y muestre el perímetro y la superficie de un círculo a partir del valor de su radio.

```
PROGRAM areaPerimetroCirculo;
```

```
CONST pi = 3.1416;
```

```
VAR radio, perim, area: real; //el radio es DE; perim y area son DS
```

```
BEGIN
```

```
(1) write('Ingrese el radio '); readln(radio);
```

```
(2) { perim := pi * 2 * radio;  
      area := pi * sqr(radio);
```

```
(3) { writeln('El perímetro es ', perim:3:3);  
      write('El Area es', area:3:3); readln;
```

```
END.
```

Por ahora asumimos que para el radio nos ingresan un valor positivo.

# Elementos de Pascal

La estructura del bloque ejecutable, como la de muchos de los programas que desarrollaremos, es:

Leer los valores de los datos de entrada (1).

Procesar y generar datos de salida a partir de datos de entrada (2).

Mostrar los valores de los datos de salida (3).

En esta materia, para los mismos valores de datos de entrada, un programa generará siempre los mismos valores para los datos de salida.



**CODIFICACIÓN**



# Elementos de Pascal

## Encabezamiento

## Bloque de declaraciones

```
PROGRAM areaPerimetroCirculo;
```

```
CONST pi = 3.1416;
```

```
VAR radio, perim, area: real;
```

```
BEGIN
```

```
  write('Ingrese el radio '); read(radio);
```

```
  perim := pi * 2 * radio;
```

```
  area := pi * sqr(radio);
```

```
  writeln ('El perimetro es ', perim);
```

```
  write('El Area es ', area);
```

```
END.
```

## Bloque ejecutable

# Elementos de Pascal

CODIFICACIÓN

Un programa en Pascal incluye tres tipos de **identificadores**:

- Reservados
- Predefinidos
- Definidos por el programador



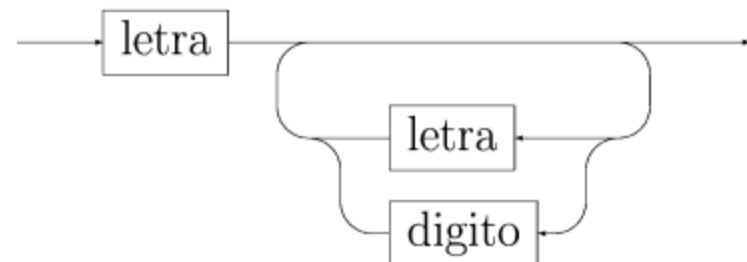
Todo identificador **comienza con una letra** y puede seguir con otras letras, dígitos o *guiones bajos*.

**Pascal no distingue mayúsculas y minúsculas.**

**(NO ES sensible a las mayúsculas)**



*identificador*



# Elementos de Pascal

## Palabras reservadas

Las palabras reservadas  
tienen una funcionalidad ESPECÍFICA en Pascal  
y el programador sólo puede usarlas para eso.

|         |        |       |           |
|---------|--------|-------|-----------|
| program | if     | of    | file      |
| uses    | then   | for   | function  |
| label   | else   | to    | procedure |
| var     | until  | array | goto      |
| const   | repeat | div   | not       |
| begin   | while  | mod   | or        |
| end     | do     | with  | and       |
| end     | do     | with  | end       |
| end     | do     | with  | end       |

# Elementos de Pascal

CODIFICACIÓN

```
PROGRAM  areaPerimetroCirculo;  
CONST  pi = 3.1416;  
VAR  radio, perim, area: real;  
  
BEGIN  
    write('Ingrese el radio ');read(radio);  
    perim := pi * 2 * radio;  
    area := pi * sqr(radio);  
    writeln('El perímetro es ', perim);  
    write('El área es', area);  
END .
```

Los identificadores **reservados** permiten especificar la estructura del programa y no pueden ser redefinidos.



# Elementos de Pascal

```
PROGRAM  areaPerimetroCirculo;  
CONST  pi = 3.1416;  
VAR  radio, perim, area:  real;  
  
BEGIN  
    write('Ingrese el radio ');read(radio);  
    perim := pi * 2 * radio;  
    area := pi * sqr(radio);  
    writeln('El perímetro es ', perim);  
    write('El área es', area);  
END.
```

Los identificadores **predefinidos** tienen un significado inicial pero el programador puede cambiarlo.



# Elementos de Pascal

```
PROGRAM    areaPerimetroCirculo;  
  
CONST  pi = 3.1416;  
  
VAR    radio, perim, area: real;  
  
BEGIN  
  
    write('Ingrese el radio ');read(radius);  
    perim := pi * 2 * radius;  
    area := pi * sqr(radius);  
    writeln('El perímetro es ', perim);  
    write('El área es', area);  
  
END.
```

Los identificadores **definidos por el programador** se declaran en el bloque de declaraciones, excepto el nombre del programa.

# Elementos de Pascal

## Variables y Constantes

```
CONST pi = 3.1416;  
VAR radio, perim, area: real;
```

- **pi** es el nombre de una constante; a lo largo de todo el programa mantiene el valor establecido en la declaración.
- El **valor** asignado a una constante (**usando el “=”**) **determina** además su **tipo**.
- En el ejemplo **pi** es de tipo real.

**CODiFiCACIÓN**

# Elementos de Pascal

## Variables y Constantes

```
CONST pi = 3.1416;  
VAR radio, perim, area: real;
```

- **perim, area y radio** son identificadores (nombres) de variables.
- Las **variables pueden cambiar de valor** y pueden tomar valores diferentes cada vez que el programa se ejecuta.
- En la **declaración** de una variable se establece su **tipo** (que determina los valores que podrá tener y las operaciones en las que podrá participar).
- **real** es un tipo de dato predefinido (como integer, boolean, etc.).

# Elementos de Pascal

## Variables y Constantes

```
CONST pi = 3.1416;  
VAR radio, perim, area: real;
```

- Una variable o constante de tipo **real** puede tomar valores dentro de un sub-conjunto de los reales y puede aparecer en expresiones aritméticas con **operadores para reales** (+, \*, /, etc).

**area div 2**

project1.lpr(5,11) Error: Incompatible types: got "Real" expected "Int64"

Error **semántico** detectado por el compilador!

errores en  
**COMPILACIÓN**

**TRUE and pi**

```
Compiling main.pas  
main.pas(7,10) Error: Operator is not overloaded: "Boolean" and "Extended"  
main.pas(18) Fatal: There were 1 errors compiling module, stopping  
Fatal: Compilation aborted
```

Error **semántico** detectado por el compilador!



# Elementos de Pascal

Posibles errores compilación

Sintáctico: Escribir mal *begin*, olvidar un ;

Semántico: No declarar la variable *lado*,  
*usar la variable lado como si fuera un dato lógico*

```
program areaCuadrado;
```

```
var lado, area: integer;
```

```
begin
```

```
  write('Ingrese la longitud del lado:');
```

```
  readln(lado);
```

```
  area := lado * lado;
```

```
  writeln('El área del cuadrado es ', area, 'cm.');
```

```
end.
```



# Elementos de Pascal

Posibles errores en ejecución:  
Ingresar una letra (aborta) en el **read**  
cuando espera un entero.

r el área  
o

Los + difíciles de detectar

Posibles errores LOGICOS:  
Usar + en vez de \*

```
program areaCuadrado;  
var lado, area: integer;  
begin
```

```
  write('Ingrese la longitud del lado:');
```

```
  readln(lado);
```

```
  area := lado * lado;
```

```
  writeln('El área del cuadrado es ', area, 'cm.');
```

```
end.
```



## TIPOS DE ERRORES según el momento en el que ocurren:

\* **COMPILACIÓN** (no compila; sintácticos TODOS y semánticos algunos)

\* **EJECUCION** (compila; aborta durante la ejecución; siempre semánticos. Nunca son errores sintácticos.)

\* **APLICACIÓN** (compila y ejecuta; no hace lo que debería; siempre semánticos – lógicos.  
Nunca son errores sintácticos.)

# Fin .SINTAxis



- Es fundamental tener clara la sintaxis durante la etapa de CODIFICACION (escribir el código fuente).
- En **COMPILACION** se detectan **TODOS** los **errores de sintaxis**, además de otros tipos de errores (los **semánticos**<sup>1</sup>).

```
programa EJE11471;  
var A B : Integer;  
begin  
  A := 5;  
  b := A * 6;  
  write(B);  
end
```

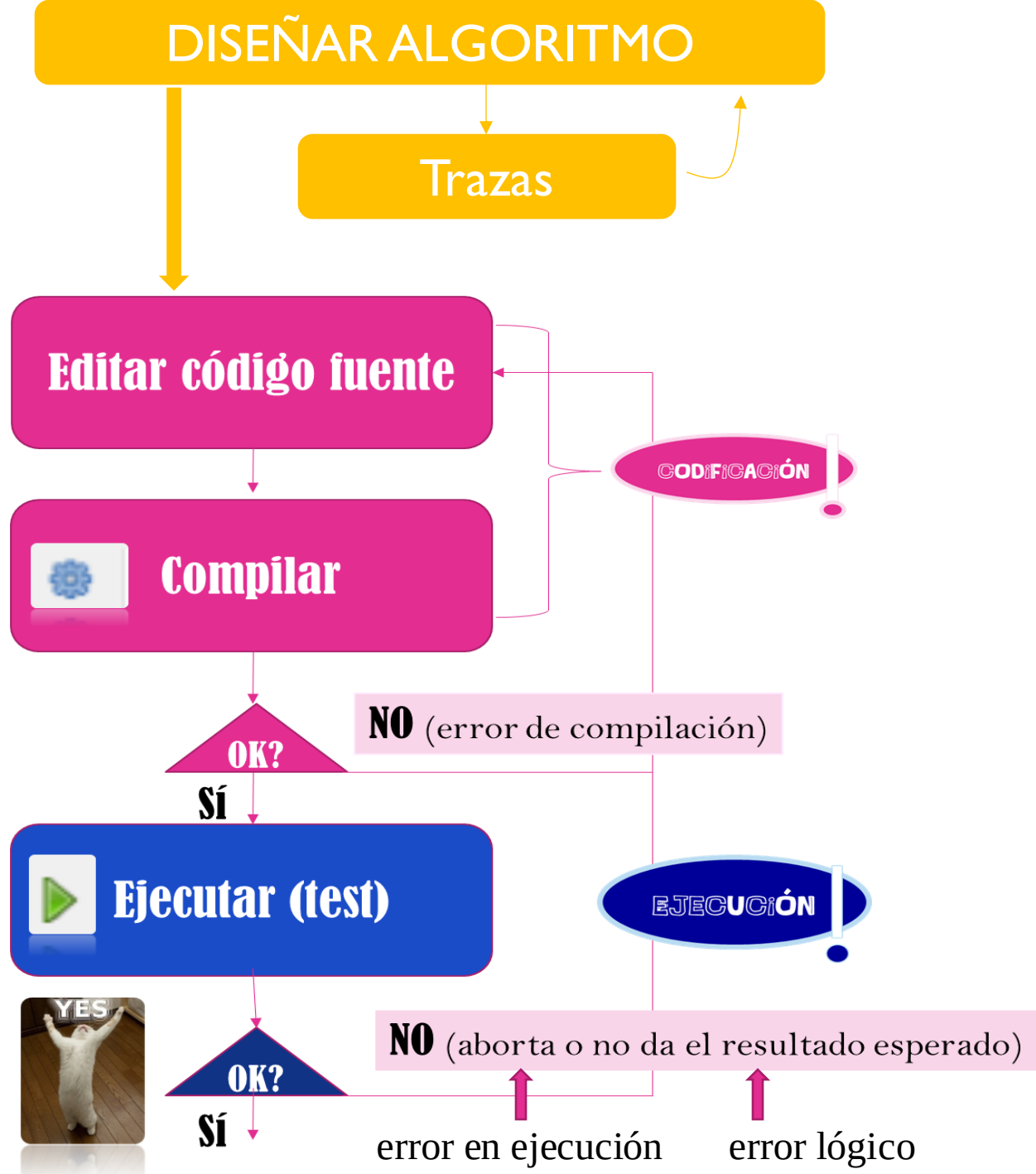
- ✗ Se escribió **programa** en vez de **program**.
- ✗ Al declarar las variables A y B, entre ellas, hay que escribir una **coma** “,”.

Tener en cuenta que los mensajes de error que da el compilador a veces no son tan claros...

<sup>1</sup>**Semánticos**: identificadores no definidos; operadores y operandos con tipos incompatibles.



# CICLO DE PROGRAMACIÓN



# CONCEPTO BASICO: SENTENCIA

Acciones

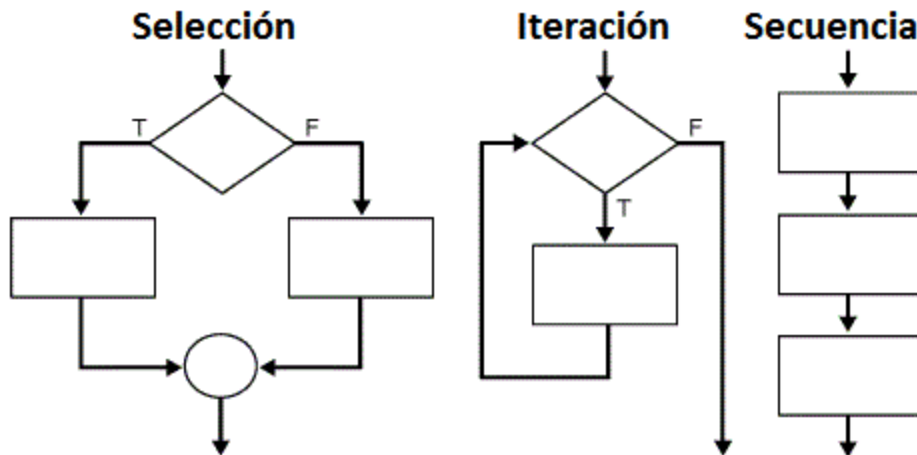
Las sentencias son los elementos básicos  
en los que se divide el código  
en un lenguaje de programación.

Un programa es un conjunto de sentencias  
que se ejecutan para realizar una cierta tarea.

Además, como ya habrás visto, en *Pascal*  
el signo que las separa es el **punto y coma**.

# ESTRUCTURAS DE CONTROL

- ✓ **Secuencia**
- ✓ **Condicionales (selección)**
- ✓ **Iteración**



Definen el orden  
las acciones

# ESTRUCTURA DE CONTROL **SECUENCIAL**

Las sentencias se ejecutan en el orden en el que se encuentran.  
No hay ambigüedad.

Dados dos datos  $x$  e  $y$ , intercambiar sus valores.



Algoritmo Intercambio

DE:  $x, y$

DS:  $x, y$

Daux: temp

$\text{temp} \leftarrow x$

$x \leftarrow y$

$y \leftarrow \text{temp}$

Es equivalente realizar  
esta secuencia de asignaciones?

$x \leftarrow x+y$

$y \leftarrow x-y$

$x \leftarrow x-y$



# ESTRUCTURA DE CONTROL **SECUENCIAL**

Problema: Convierta una cantidad de tiempo (en segundos), en el tiempo correspondiente expresado en horas, minutos y segundos.

3817 segundos = 1 horas, 3 minutos y 37 segundos

¿Cuántos datos de entrada hay? ¿Cuáles son?

¿Cuántos datos de salida hay?

¿Hay alguna restricción que debemos controlar sobre los datos de entrada?



# ESTRUCTURA DE CONTROL **CONDICIONAL**

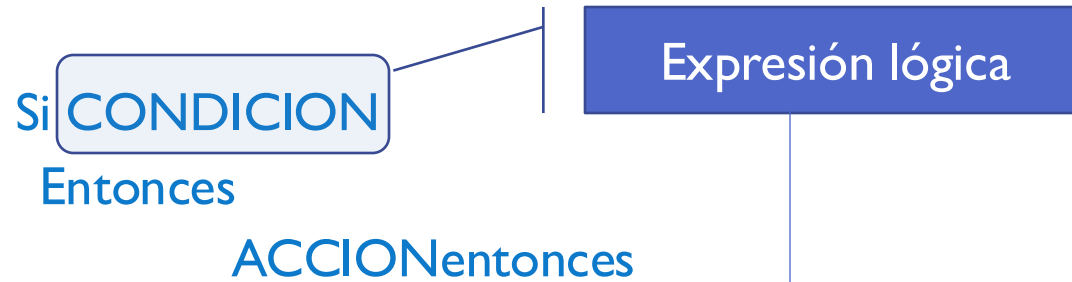
Las **estructuras de control condicionales** permiten modelar situaciones en las cuales la **ejecución** de una acción o de un bloque de acciones **depende de una condición**.

Utilizamos el condicional cuando:

- La ejecución de una sentencia **depende** de una condición.
- Dos grupos de sentencias son **mutuamente excluyentes** según una condición.

# ESTRUCTURA DE CONTROL **CONDICIONAL**

Forma de expresarlo en pseudocódigo:

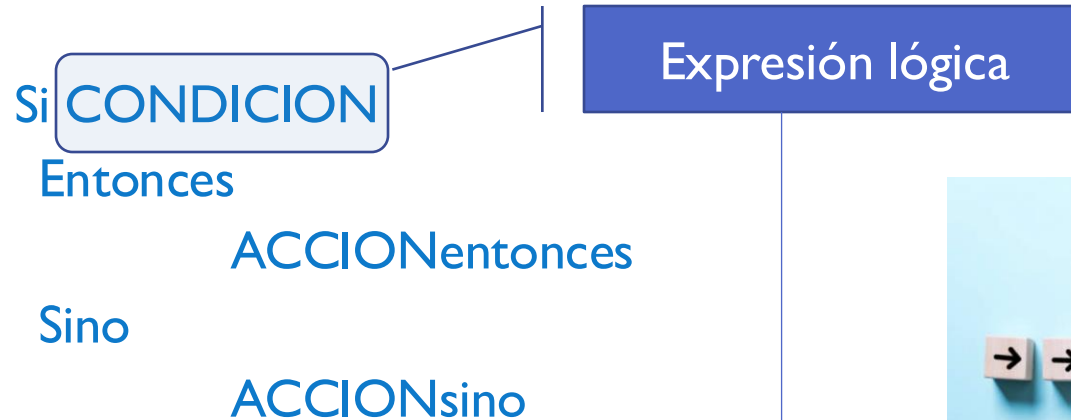


Forma de expresarlo en Pascal:

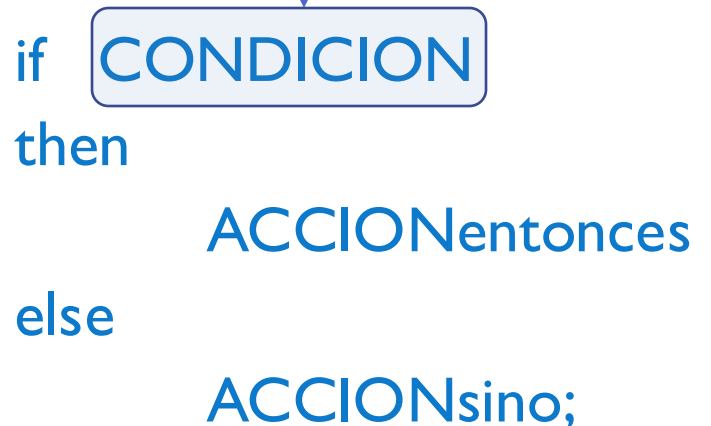


# ESTRUCTURA DE CONTROL **CONDICIONAL**

Forma de expresarlo en pseudocódigo:



Forma de expresarlo en Pascal:



No va punto y coma!  
(antes del ELSE)



# CONDICIONAL: IF

Máximo

A partir de dos números, **obtener** el máximo entre los dos.

Algoritmo maximo

DE: x, y

DS: max

Si  $(x > y)$

entonces

$\text{max} \leftarrow x$

sino

$\text{max} \leftarrow y$

```
program maximo;
```

```
var x, y, max: integer;
```

```
begin
```

```
  write ('Ingrese dos números: ');
```

```
  readln (x, y);
```

```
  if (x > y) then max:=x
```

```
    else max:=y;
```

```
  write ('El maximo es ', max);
```

```
end.
```

- dos instrucciones o bloques son mutuamente excluyentes según una

# CONDICIONAL: IF

CODIFICACIÓN

Escribir un programa en Pascal que reciba un valor de radio, y luego pregunte al usuario si quiere que se calcule el perímetro o la superficie, y muestre el resultado correspondiente.

# CONDICIONAL: IF

```
program opciones;  
const PI = 3.1415;  
var opcion: integer;  
    radio, perim, area: real;  
  
begin  
    writeln('Ingrese la longitud del radio en cm: ');  
    readln(radio);  
    writeln('Ingrese 1 para calcular perimetro, y 2 para calcular area: ');  
    readln(opcion);  
    if (opcion = 1)  
    then  
        begin  
            perim := PI * 2 * radio;  
            writeln('El perimetro es: ', perim:0:2, ' cm.');        end  
    else  
        begin  
            area := PI * sqr(radio);  
            writeln('El area es: ', area:0:2, ' cm2.');        end;  
    readln;  
end.
```

# CONDICIONAL: IF

Dado un precio, calcular el descuento (según tabla) y devolver el nuevo precio.

Algoritmo **Descuento**

DE: precio

DS: precioConDescuento

Daux: descuento

Si (precio > 100)

entonces

descuento  $\leftarrow$  20

sino

Si (precio < 15)

entonces

descuento  $\leftarrow$  0

sino

descuento  $\leftarrow$  10

precioConDescuento  $\leftarrow$  precio - descuento

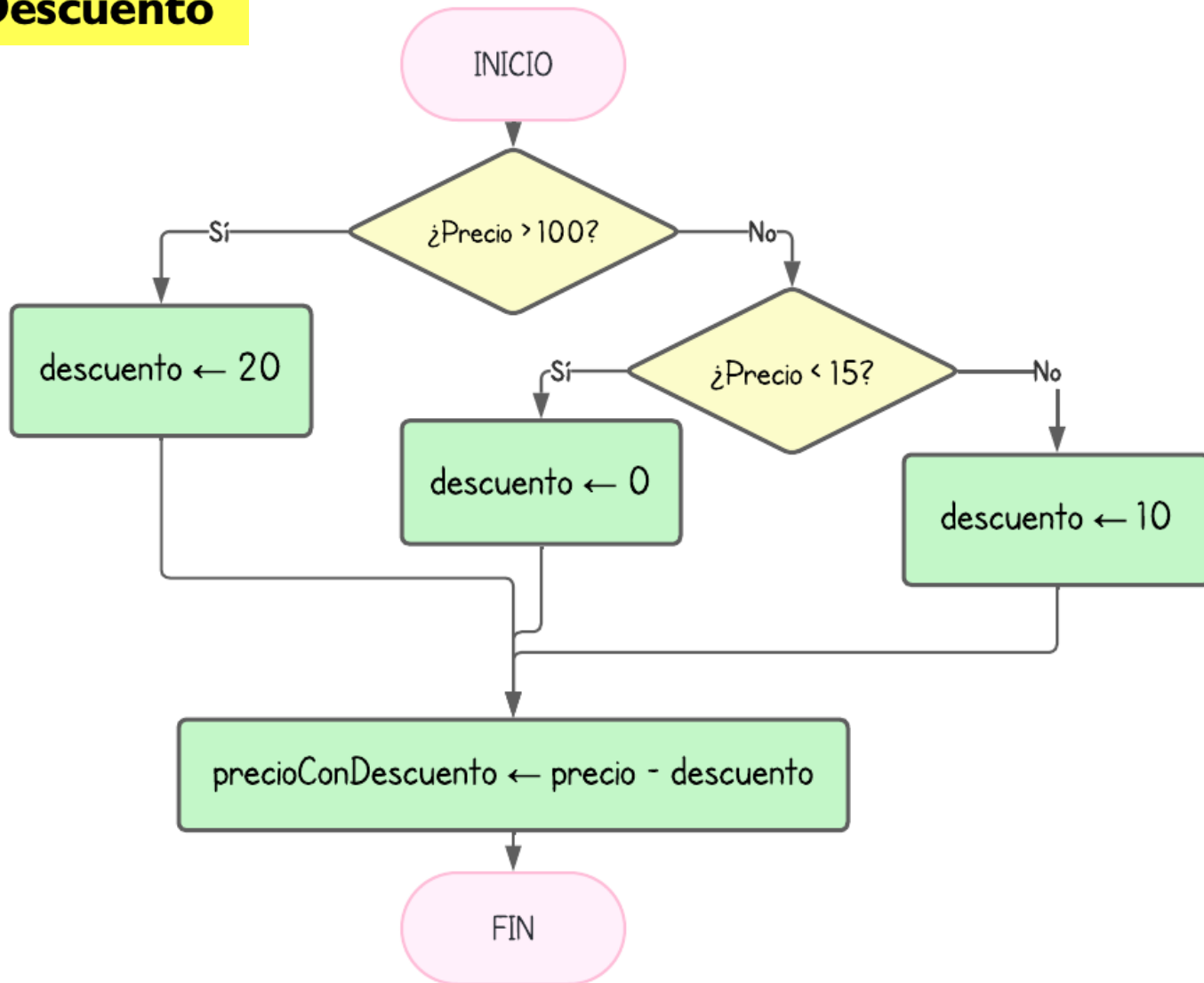
| precio        | Descuento |
|---------------|-----------|
| Mayor que 100 | 20\$      |
| [15,100]      | 10\$      |
| Menor a 15    | 0\$       |

En qué caso el  
descuento será  
de 10 pesos?

**Pasar  
a PASCAL**

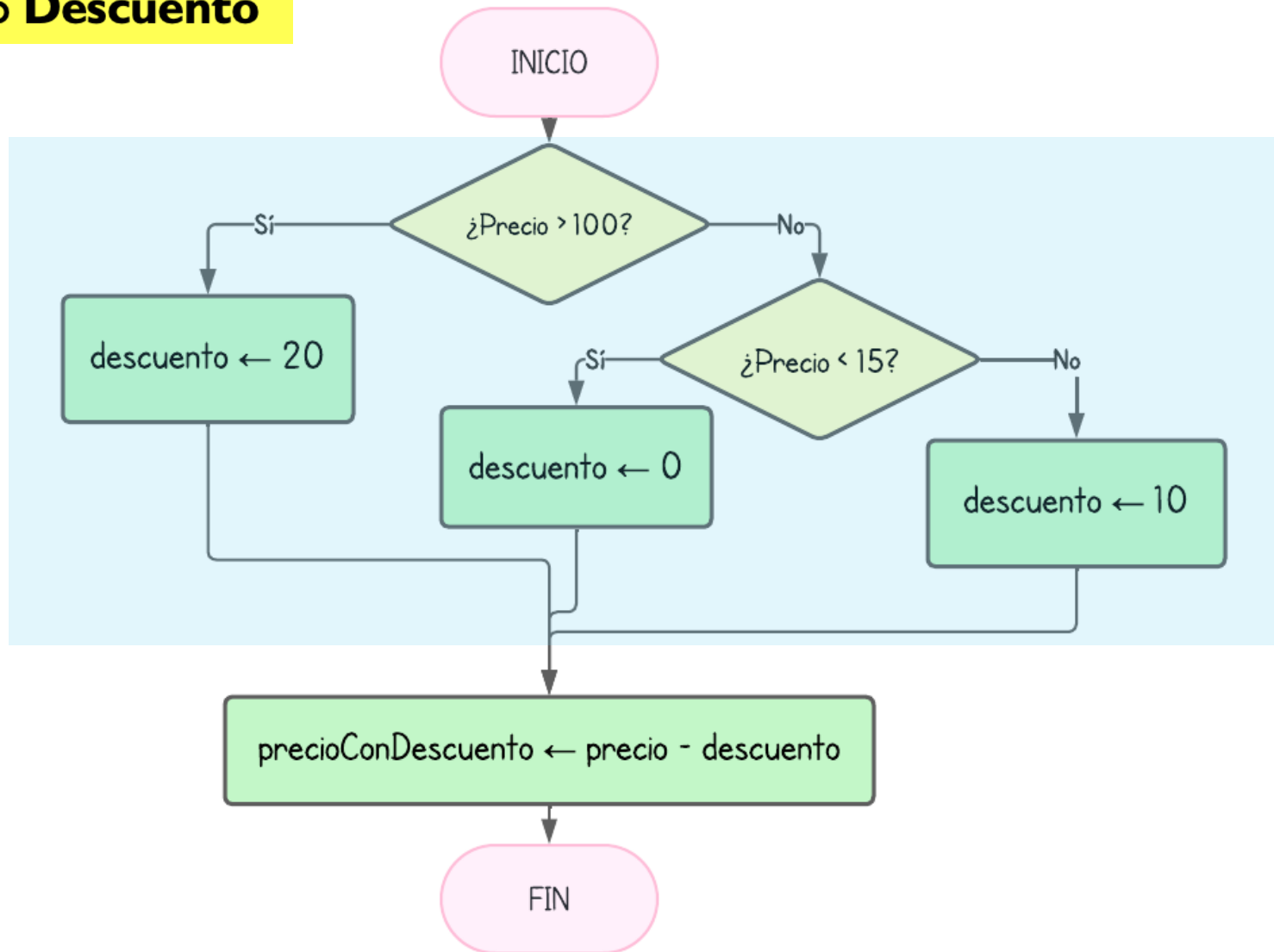
# CONDICIONAL: IF

## Algoritmo Descuento



# CONDICIONAL: IF

## Algoritmo Descuento



# CONDICIONAL: IF

Dado un precio, calcular el descuento (según tabla) y devolver el nuevo precio.

Tres escenarios posibles en EJECUCIÓN:

Primero:

```
descuento ← 0  
precioConDescuento ← precio - descuento
```

Segundo:

```
descuento ← 10  
precioConDescuento ← precio - descuento
```

Tercero:

```
descuento ← 20  
precioConDescuento ← precio - descuento
```

| precio        | Descuento |
|---------------|-----------|
| Mayor que 100 | 20\$      |
| [15,100]      | 10\$      |
| Menor a 15    | 0\$       |

# CONDICIONAL: IF

Dado un precio, calcular el descuento (según tabla) y devolver el nuevo precio.

## Algoritmo **Descuento2**

DE: precio

DS: precioConDescuento

Daux: descuento

Si (precio > 100)

entonces

descuento  $\leftarrow$  20

Si (precio  $\geq$  15) Y (precio  $\leq$  100)

entonces

descuento  $\leftarrow$  10

Si (precio < 15)

entonces

descuento  $\leftarrow$  0

precioConDescuento  $\leftarrow$  precio - descuento

| precio        | Descuento |
|---------------|-----------|
| Mayor que 100 | 20\$      |
| [15,100]      | 10\$      |
| Menor a 15    | 0\$       |

¿Los escenarios posibles  
en ejecución  
son los mismos?



# CONDICIONAL: IF

```
program Descuento;  
var precio, desc, precioConDesc: integer;  
begin  
  writeln('Ingrese el precio del producto:');  readln(precio);  
  if (precio > 100)  
  then  
    desc := 20  
  else  
    if (precio < 15)  
    then  
      desc := 0  
    else  
      desc := 10;  
  precioConDesc := precio - desc;  
  writeln('El precio luego de aplicar el descuento es:', precioConDesc);  
end.
```

**SENTENCIA SIMPLE:**  
Una **única acción**  
condicionada por una parte  
de la estructura de control

Escribir un programa para que dado el precio de un producto se calcule y se muestre el descuento (de acuerdo a la tabla) y se muestre también el nuevo precio luego de aplicar el descuento.

```
program descuentoCompuestas;
var precio, desc, precioConDesc: integer;
begin
  writeln('Ingrese el precio del producto:'); readln(precio);
  if (precio > 100)
  then
    begin
      desc:=20;
      writeln('El descuento es de 20 pesos.');
```

```
    end
  else
    if (precio < 15)
    then
      begin
        desc:=0;
        writeln('No tiene descuento con este producto.');
```

```
      end
    else
      begin
        desc:=10;
        writeln('El descuento es de 10 pesos.');
```

```
      end;
    precioConDesc:=precio-desc;
    writeln('El precio luego de aplicar el descuento es:', precioConDesc);
  end.
```

| precio        | Descuento |
|---------------|-----------|
| Mayor que 100 | 20\$      |
| [15,100]      | 10\$      |
| Menor a 15    | 0\$       |

**SENTENCIA COMPUESTA:**  
dos o más acciones  
condicionadas por una parte  
de la estructura de control

# CONDICIONAL: IF

La estructura de control **if** evalúa una **expresión lógica**.

Esta expresión le permite al programador establecer una **condición**.

- Si la expresión lógica computa **true**, la condición se satisface, y se ejecutan la instrucciones que siguen a la palabra reservada **then**.
- Si la condición es **false**, se ejecutan las instrucciones que siguen a la palabra reservada **else**.

Debemos **verificar** por lo menos con dos valores: uno con el que la condición se evalúe a *true* y el otro que evalúe a *false*.

# .SINTAXIS



Aunque generalmente una sentencia debe ir seguida de un **punto y coma** para separarla de la siguiente, existen casos en los que no es necesario incluirlo.

Y estos casos se dan cuando después de la sentencia hay una **palabra reservada** que delimita de alguna forma un trozo de código.

Por ejemplo end, else y until.

El ; antes de un **end** es opcional (Pascal lo permite).

Si usamos ; antes de un **else** o un **until**, Pascal dará un error en compilación.

```
begin
    writeln('Hola');
    writeln('Chau')
end.
```

equivalentes

```
begin
    writeln('Hola');
    writeln('Chau');
end.
```

```
begin
...
    if (a=0)
        then
            write('CERO')
        else
            write('No es cero');
end.
```

correcto

```
begin
...
    if (a=0)
        then
            write('CERO');
        else
            write('No es cero');
end.
```

error en  
compilación

Ventana de Mensajes

project1.lpr(5,3) Fatal: Syntax error, ";" expected but "ELSE" found

Corolario: RECORDAR SIEMPRE QUE **ANTES DEL ELSE**  
NO VA PUNTO Y COMA.

# CONDICIONAL: IF

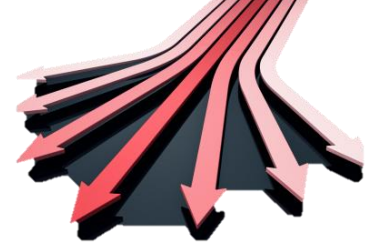
Escribir un programa en Pascal que reciba un valor de radio, y luego pregunte al usuario si quiere que se calcule el diámetro (opción 1), el perímetro (opción 2) o la superficie (opción 3), y muestre el resultado correspondiente.

Hacer dos trazas. Una para un radio de 2 cm., para calcular el área. Otra para un radio de 3.5 cm., para calcular el diámetro.



A trabajar

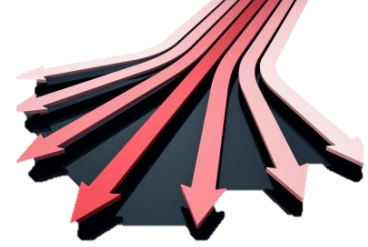
# CONDICIONAL *ESPECIAL*: CASE



Se utiliza cuando una o varias instrucciones **mutuamente excluyentes** dependen de una **expresión** de un tipo **escalar**.

En caso de que la <expresión o variable> valga  
<valor o valores>: <acción o acciones>;  
<valor o valores>: <acción o acciones>;  
<valor o valores>: <acción o acciones>;  
...

# CONDICIONAL *ESPECIAL*: CASE



Se utiliza cuando una o varias instrucciones **mutuamente excluyentes** dependen del valor de **variable o expresión** de un tipo **escalar**.

Se evalúa una **expresión escalar** y de acuerdo al valor computado se selecciona la instrucción que se va a ejecutar.

Escalar: es un tipo no estructurado en el que está definida una relación de orden, con un siguiente y un anterior.

**Integer** y **char** son escalares.



# CONDICIONAL: CASE



PROBLEMA: A partir de una nota numérica, mostrar el concepto.

| NOTA     | Concepto  |
|----------|-----------|
| 0        | Malo      |
| 1, 2 o 3 | Regular   |
| 4, 5 o 6 | Bueno     |
| 7, 8 o 9 | Muy bueno |
| 10       | Excelente |

El cartel que se muestra no depende del resultado de una pregunta por SI o por NO, es decir, no depende de una CONDICION.

Algoritmo concepto

DE: **nota** (entero entre 0 y 10)

DS: concepto (texto por pantalla)

En caso de que **nota** valga:

0: Mostrar MALO

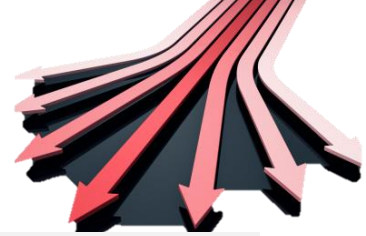
1, 2, 3: Mostrar REGULAR

4, 5, 6: Mostrar BUENO

7, 8, 9: Mostrar MUY BUENO

10: Mostrar EXCELENTE

# CONDICIONAL *ESPECIAL*: CASE

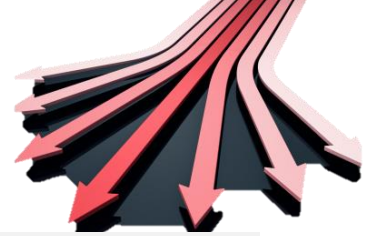


```
CASE <expresion> OF
  <lista_opciones>: <una sentencia simple o compuesta>;
  <lista_opciones>: <una sentencia simple o compuesta>;
  <lista_opciones>: <una sentencia simple o compuesta>;
  ...
END;
```

Pascal  
programming

- <expresion> es cualquier variable o expresión de tipo **ordinal** (usamos *integer* y *char*)
- <lista\_opciones> puede ser:
  - un valor individual ej: 2
  - una secuencia de valores individuales ej: 'a','e','i','o' o 10, 15, 20
  - intervalos ej: 1..100, 'a'..'z'
  - una combinación de los anteriores separados con comas.
- <una sentencia simple o compuesta> puede ser una sentencia simple o compuesta (encerrada con *begin* y *end*)

# CONDICIONAL *ESPECIAL*: CASE



```
CASE <expresion> OF
  <lista_opciones>: <una sentencia simple o compuesta>;
  <lista_opciones>: <una sentencia simple o compuesta>;
  <lista_opciones>: <una sentencia simple o compuesta>;
  ...
END;
```

Pascal  
programming

<expresion> y <lista\_opciones> deben ser del mismo tipo.

- 1) Se evalúa <expresion> y se obtiene un **valor**
- 2) Se busca (de arriba hacia abajo) si **valor** está en una de las <lista\_opciones>
- 3) Si se encuentra el **valor** se ejecuta la sentencia siguiente al “:” y luego sigue inmediatamente después del END;
- 4) Si **valor** no pertenece a ninguna de las <lista\_opciones> no se ejecuta ninguna sentencia.

# CONDICIONAL: CASE



**PROBLEMA:** A partir de una nota numérica, mostrar el concepto.

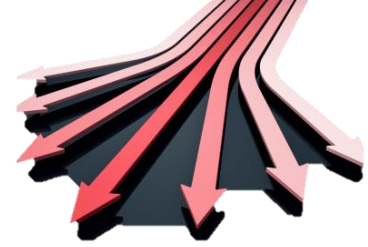
| NOTA     | Concepto  |
|----------|-----------|
| 0        | Malo      |
| 1, 2 o 3 | Regular   |
| 4, 5 o 6 | Bueno     |
| 7, 8 o 9 | Muy bueno |
| 10       | Excelente |

Subrango de los enteros  
(desde 4 hasta 6)

Que pasa si  
ingresan un valor  
que no esta entre  
0 y 10?

```
program Concepto;  
var nota: integer;  
begin  
  writeln('Ingrese la nota:'); read(nota);  
  case nota of  
    0: writeln('Malo');  
    1, 2, 3: writeln('Regular');  
    4..6: writeln('Bueno');  
    7, 8, 9: writeln('Muy Bueno');  
    10: writeln('Excelente');  
  end;  
  readln;  
end.
```

# CONDICIONAL: CASE



Escribir un programa que lea una expresión aritmética binaria en notación prefija y la compute. En la notación prefija el operador +, -, \* o / precede a sus operandos.

¿Cuáles son los datos de entrada?

+ 31  
8

¿Cuáles son los datos de salida?

¿Cuáles pueden ser los casos de prueba?



# CONDICIONAL: CASE

Escribir un programa que lea una expresión aritmética binaria en notación prefija y la compute.

Algoritmo Evaluar

DE: op (char), x (ent), y (ent)

DS: sol (real)

En caso de que **op** valga:

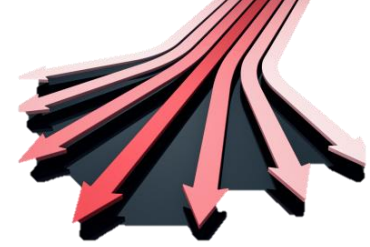
‘+’: sol  $\leftarrow$  x + y

‘-’: sol  $\leftarrow$  x - y

‘\*’: sol  $\leftarrow$  x \* y

‘/’: sol  $\leftarrow$  x / y

# CONDICIONAL: CASE



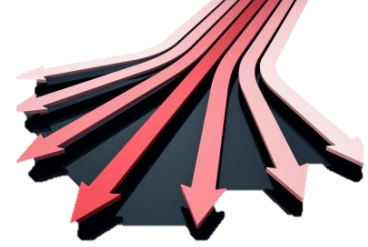
```
program evaluaExpresion;
```

```
var
```

```
  x, y: integer; sol: real;  op: char;
```

```
...
```

# CONDICIONAL: CASE

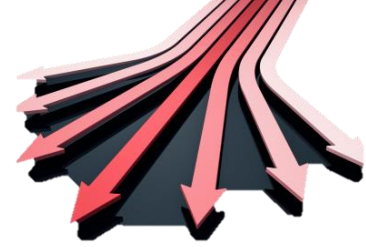


```
write ('Ingrese operador operando operando');  
readln(op, x, y);  
if (op <> '+' and (op <> '-') and  
    (op <> '*') and (op <> '/'))  
then  
    writeln ('El operador no existe')  
else  
    begin  
        ...  
    end
```

Pascal  
programming



# CONDICIONAL: CASE



...begin

write ('Ingrese operador operando operando ');

readln(op, x, y);

if (op<>'+' ) and (op<>'-' ) and (op<>'\*' ) and (op<>'/' )

then writeln ('El operador no existe')

else

**begin** {realizamos la operación sabiendo que es valido el operador}

case op of

'+' : sol := x+y;

'-' : sol := x-y;

'\*' : sol := x\*y;

'/' : sol := x / y;

end; {del case, no tiene begin}

writeln ('La solucion es ',sol);

Sentencia compuesta en el else:  
Una sentencia case  
y una sentencia writeln.

**end;**

**end.**

# EMPEZAMOS A ESTUDIAR SOLOS...



# Elementos de Pascal

ENCABEZAMIENTO

DECLARACIONES

BLOQUE

BLOQUE EJECUTABLE

# Elementos de Pascal

**BEGIN**

```
write('Ingrese el radio ');
```

```
read(radius);
```

```
perim := pi * 2 * radius;
```

```
area := pi * radius * radius;
```

```
writeln('El perímetro es ',perim);
```

```
write('El Area es ',area);
```

**END.**

**write** es un procedimiento predefinido.

La primera instrucción **write** muestra un mensaje para indicarle al usuario que el programa va a quedar esperando que ingrese la longitud del radio.

# Elementos de Pascal

BEGIN

```
write('Ingrese el radio ');
```

```
read(radius);
```

```
perim := pi * 2 * radius;
```

```
area := pi * radius * radius;
```

```
writeln('El perímetro es ', perim);
```

```
write('El Area es ', area);
```

END.

La segunda y la tercera instrucciones **write** permiten que el programa le muestre al usuario un cartel y el valor que está almacenado en una variable.

# Elementos de Pascal

BEGIN

```
write ('Ingrese el radio ');
```

```
read (radio);
```

```
perim := pi * 2 * radio;
```

```
area := pi * radio * radio;
```

```
writeln ('El perímetro es ', perim);
```

```
write ('El Area es ', area);
```

END.

**read** es un procedimiento predefinido.

La instrucción **read** permite que el usuario ingrese el valor y que el mismo quede almacenado en la variable **radio**.

# Elementos de Pascal

BEGIN

```
write ('Ingrese el radio ');read(radius);
```

```
perim := pi * 2 * radius;
```

```
area := pi * radius * radius;
```

```
writeln ('El perímetro es ',perim);
```

```
write('El Area es ',area);
```

END.

La Ira instrucción de asignación se ejecuta en **dos pasos**:

- Computa la expresión  $pi * 2 * radius$
- Almacena el valor computado en la variable `perim`

La expresión consta de operadores y operandos.

# Elementos de Pascal

BEGIN

```
write ('Ingrese el radio ');read(radius);
```

```
perim := pi * 2 * radius;
```

```
area := pi * radius * radius;
```

```
writeln ('El perímetro es ',perim);
```

```
write('El Area es ',area);
```

END.

La 2da asignación se ejecuta en **dos pasos**:

- Computa la expresión  $\pi * \text{radius} * \text{radius}$
- Almacena el valor computado en la variable `area`

La expresión consta de operadores y operandos.



# CONDICIONAL: IF

```
1 IF (MiCondición) THEN
2   MiSentencia;
3
4 -- Esta línea ya no está dentro del IF()
5 -- Y esta línea tampoco está dentro del IF()
```

sentencia simple

```
1 IF (MiCondición) THEN BEGIN
2   MiSentencia1;
3   MiSentencia2;
4   MiSentencia3;
5   --
6   MiSentencian;
7 END
8 -- Esta línea ya no está dentro del IF()
9 -- Y esta línea tampoco está dentro del IF()
```

sentencia compuesta

# CONDICIONAL: IF

```
1  IF (MiCondición) THEN BEGIN
2      MiSentencia1;
3      MiSentencia2;
4      MiSentencia3;
5      --
6      MiSentenciam;
7  END ELSE
8      MiSentenciax;
9  -- Esta línea ya no está dentro del ELSE
10 -- Y esta línea tampoco está dentro del ELSE
```

SENTENCIA SIMPLE

```
1  IF (MiCondición) THEN BEGIN
2      MiSentencia1;
3      MiSentencia2;
4      MiSentencia3;
5      --
6      MiSentenciam;
7  END ELSE BEGIN
8      MiSentenciax;
9      MiSentenciay;
10     MiSentenciaz;
11     --
12     MiSentenciam;
13 END
14 -- Esta línea ya no está dentro del ELSE
15 -- Y esta línea tampoco está dentro del ELSE
```

SENTENCIA COMPUESTA  
Desde “MiSentenciax”  
hasta “MiSentenciam”  
dependen del ELSE

# CONDICIONAL: IF

Leer los límites de un intervalo cerrado  $[a, b]$  y un número  $x$  y determinar si  $x$  está en el intervalo.

$x$  está en el intervalo  $[a, b]$  si  $a \leq x \leq b$

Pascal no admite una expresión como  $a \leq x \leq b$

$(x \geq a) \text{ and } (x \leq b)$

$(a \leq x) \text{ and } (x \leq b)$

# CONDICIONAL: IF

Leer los límites de un intervalo cerrado  $[a, b]$   
y un número  $x$  y determinar si  $x$  está en el intervalo.

¿Cuáles son los datos de entrada?

¿Cuáles son los datos de salida?

¿Cuáles pueden ser los casos de prueba?

# CONDICIONAL: IF

Algoritmo intervalo

DE: a, b, x (enteros)

DS: pertenece (lógico)

COMIENZO

Si  $((a \leq x) \text{ and } (x \leq b))$

    pertenece  $\leftarrow$  verdadero

sino

    pertenece  $\leftarrow$  falso

FIN

# CONDICIONAL: IF

Algoritmo intervalo

DE: a, b, x

DS: pertenece

COMIENZO

pertenece  $\leftarrow (a \leq x) \text{ and } (x \leq b)$

FIN

# CONDICIONAL: IF

```
program intervalo;
```

```
var a, b, x: integer; pertenece: boolean;
```

```
begin
```

```
  writeln ('Ingrese los límites del intervalo ');
```

```
  readln (a, b);
```

```
  writeln ('Ingrese el valor de x:');
```

```
  readln (x);
```

```
  pertenece:= (a<=x) and (x<=b);
```

```
  if
```

```
    pertenece
```

```
    then writeln ('x pertenece al intervalo')
```

```
    else writeln ('x no pertenece al intervalo');
```

```
end.
```

# CONDICIONAL: CASE

Escribir un programa que lea un carácter y muestre un cartel indicando si se trata de una letra mayúscula, minúscula, un dígito u otro símbolo.

```
*caseSimb.lpr
1  program caseSimb;
.  var ch: char;
.  begin
.    writeln('Ingrese un simbolo del teclado: ');
5    readln(ch);
.    case ch of
.      'a'..'z': writeln('Es una letra minuscula');
.      'A'..'Z': writeln('Es una letra mayuscula');
.      '0'..'9': writeln('Es un digito');
10   else writeln('Es un simbolo distinto de letra y digito');
.    end;
.    readln;
.  end.
```

Pascal permite establecer **subrangos** de valores de tipo integer o char.



# CONDICIONAL: CASE



PROBLEMA: Dada una nota numérica, mostrar la letra correspondiente.

```
program pepe;  var x: integer; begin
writeln('Ingrese la calificacion del alumno: ');readln(x);
  case x of 90..100: writeln('A');80..89: writeln('B'); 70..79:
writeln('C');60..69:writeln('D');0..59:writeln('C');end;readln;end.
```



# CONDICIONAL: CASE



PROBLEMA: Dada una nota numérica, mostrar la letra correspondiente.

```
VAR Calificacion : Integer;  
  
BEGIN  
  Write('Ingrese la Calificacion del alumno :');  
  Readln(Calificacion);  
  CASE Calificacion OF  
    90..100 : Writeln('A');  
    80..89  : Writeln('B');  
    70..79  : Writeln('C');  
    60..69  : Writeln('D');  
    0..59   : Writeln('E')  
  END;  
END.
```



# Claridad en nuestro código

- ✖ Es importante que un algoritmo (y luego el programa) sea claro, y que no presente ambigüedades para quien lo lea.
- ✖ Para lograr esto hay que tener en cuenta una serie de pautas de “buen estilo” para la realización de algoritmos:
  - Líneas para agrupar en bloques
  - Indentación
  - Uso de nombres representativos



# Ejercicio

Escriba un programa que lea un número entero y muestre carteles para indicar si:

- Es múltiplo de 2
- Es múltiplo de 3 y 5
- No es múltiplo de 7 ni de 11.
- Es múltiplo de 5 y no es múltiplo de 7.



# Ejercicio

A partir de un entero ingresado por el usuario, se desea mostrar el cartel correspondiente:

## SI N ES:

Menor que cero

0

1

2

3

4

Entre 5 y 10

Mayor a diez

## MOSTRAR:

“NEGATIVO”

“CERO”

“UNO”

“DOS”

“TRES”

“CUATRO”

“CINCO SEIS SIETE OCHO NUEVE O DIEZ”

“MAYOR A 10”

